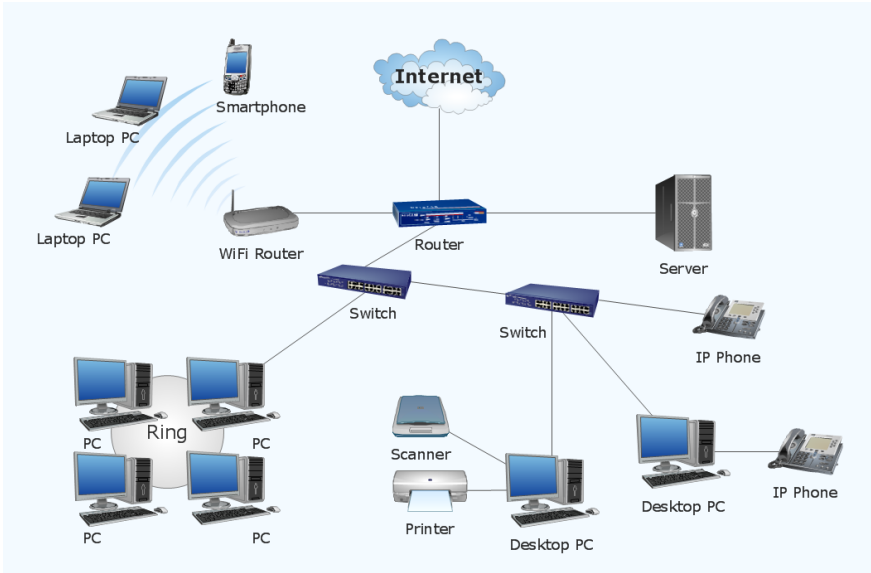
Sisteme și algoritmi distribuiți
Curs 1






Internet
Smartphone
Laptop PC
Laptop PC
WiFi Router
Router
Server
Switch
Switch
IP Phone
Ring
PC
PC
PC
PC
Scanner
Printer
Desktop PC
Desktop PC
IP Phone



Plan materie

1. Introducere. Modele și arhitecturi SD.
2. Comunicație în SD. Ceasuri logice.
3. **Sisteme sincrone**
    - 3.1 Algoritmi OSD: alegere lider, consens, sincronizare
    - 3.2 Defecte
    - 3.3 Consens și alte probleme numerice: medie, evaluare funcții, sisteme liniare
    - 3.4 Toleranță la defecte
4. **Sisteme asincrone**
    - 4.1 Organizarea SDa: teorema imposibilitate, consens relaxat
    - 4.2 Medie și sisteme liniare în context asincron
    - 4.3 Toleranță la defecte
5. **Algoritmi distribuiți aleatori**:
    - 5.1 Paradigma gossip
    - 5.2 Consens și alte probleme
    - 5.3 Toleranță la defecte

Organizare

Curs 2h/săptămână: A. P.
Laborator 2h/ săptămână: Răzvan Anghel

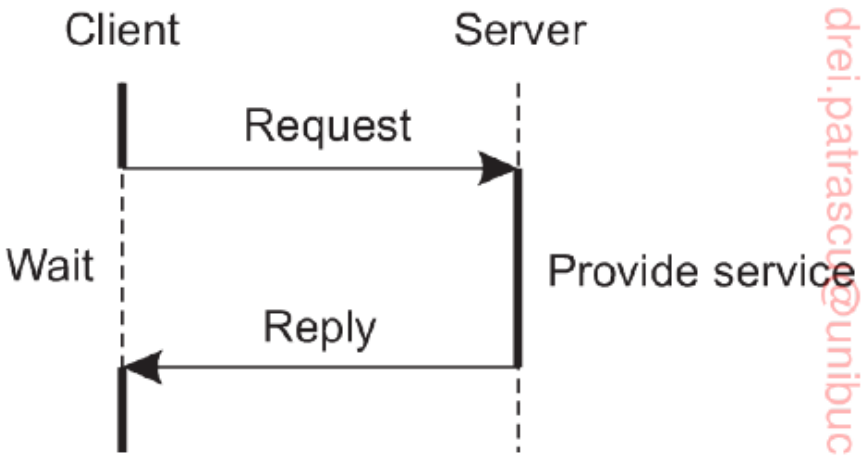
Evaluare:

1. Examen scris (50%)
2. Teme laborator (50%)

Promovare: nota 5 examen + nota 5 laborator.
Materiale și comunicare: platforma Teams.
Adresa e-mail: andrei.patrascu@fmi.unibuc.ro,
marius-iulian.mihailescu@unibuc.ro

Arhitecturi: Client-Server

- Un nod/proces are calitatea de client sau server
- Server = entitate care oferă un serviciu
- Client = entitate care apelează un serviciu

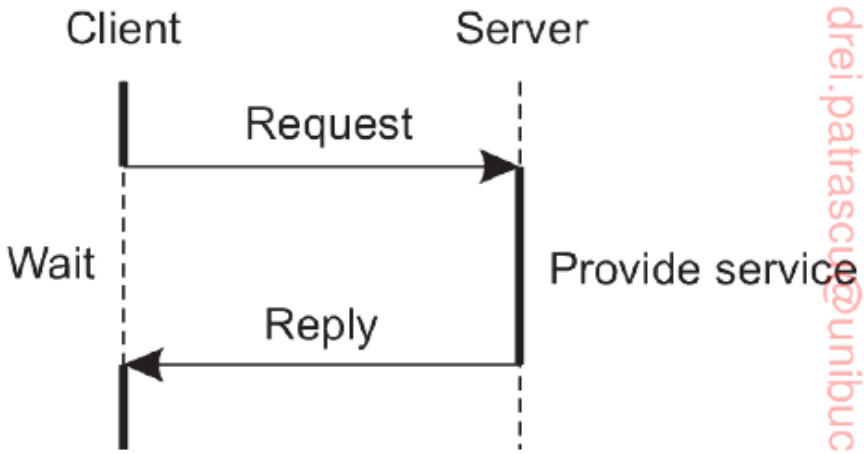


Exemple:

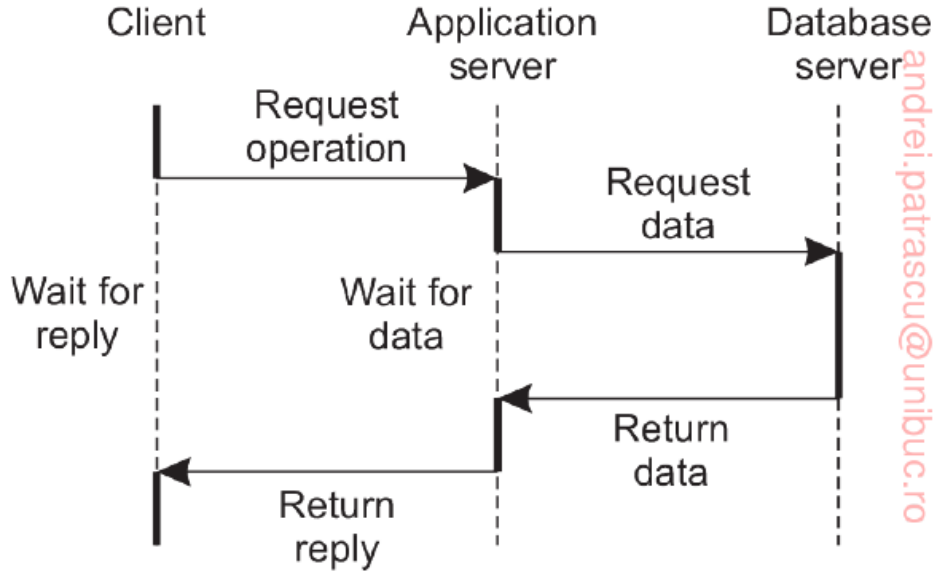
- Microsoft Teams
- Overleaf

drei.patrascu@unibuc

Arhitecturi: Client-Server



drei.patrascu@unibuc

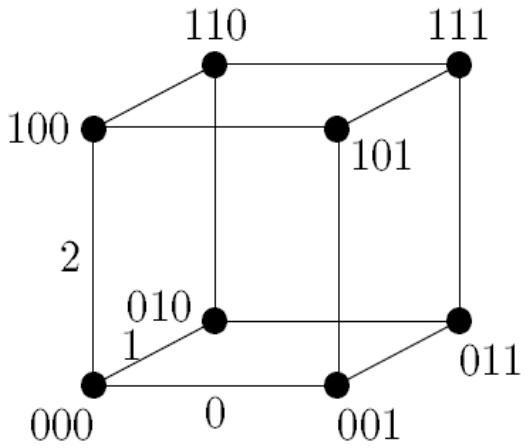


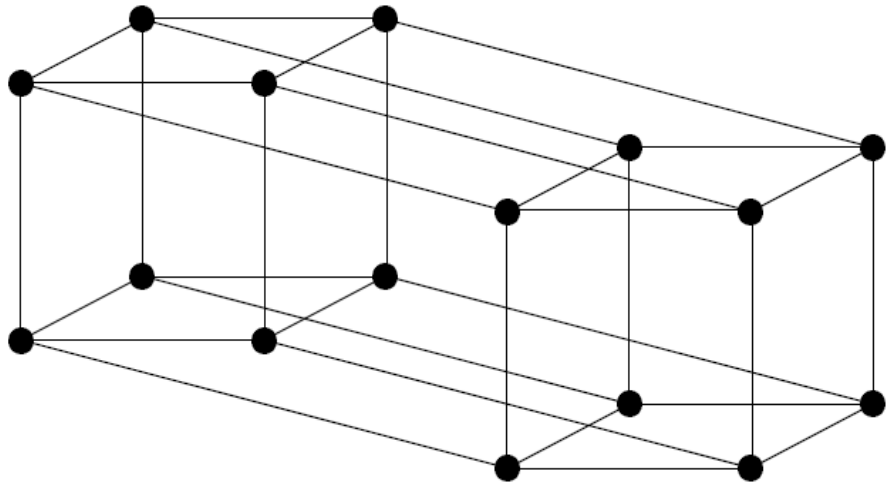
andrei.patrascu@unibuc.ro

Arhitecturi: Peer-to-peer structurate

Cluster (parallel) computing:

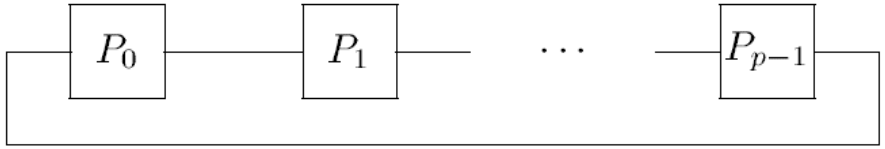
- Fiecare nod are aceeași capacitate de calcul
- Comunicație sigură între procese
- Identificator unic per proces (nod)



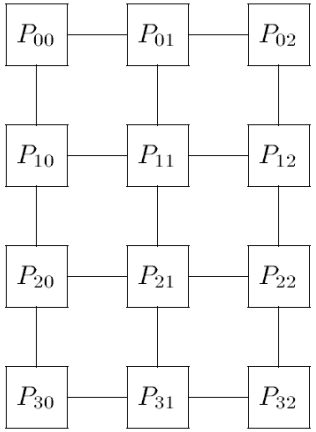


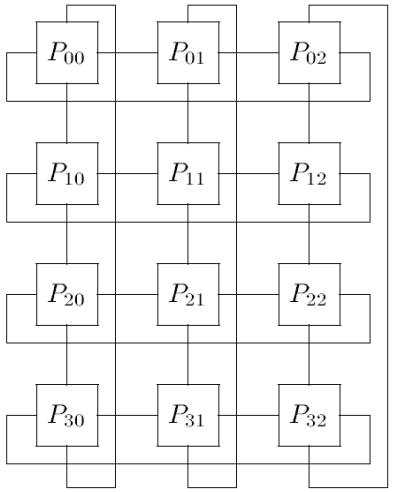
Arhitecturi: Peer-to-peer structurate

Inel

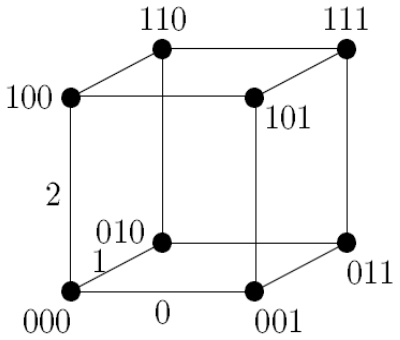


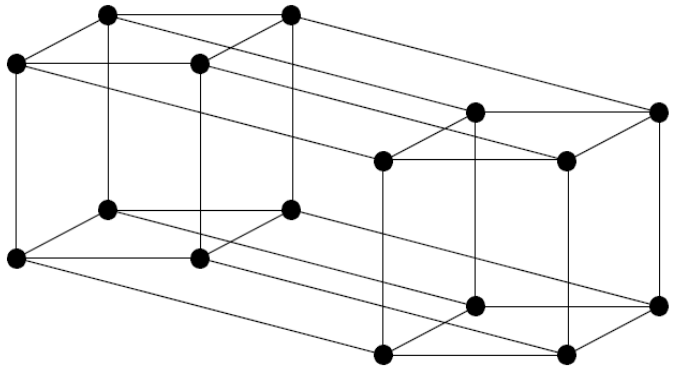
Grilă/Tor





Hipercub





Peer-to-peer structurat: El Capitan



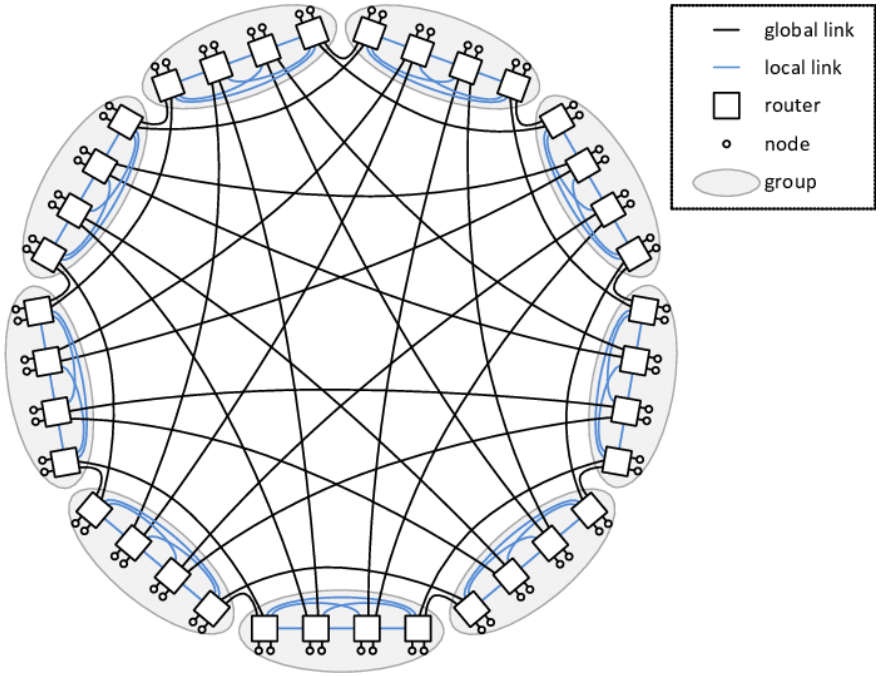
El Capitan: DOE/NNSA/LLNL
No.1 in Nov 2024

Rank	System	Cores	Rmax (PFlop/s)	Rpeak (PFlop/s)	Power (kW)
1	**El Capitan** - HPE Cray EX255a, AMD 4th Gen EPYC 24C 1.8GHz, AMD Instinct MI300A, Slingshot-11, TOSS, HPE DOE/NNSA/LLNL United States	11,039,616	1,742.00	2,746.38	29,581
2	**Frontier** - HPE Cray EX235a, AMD Optimized 3rd Generation EPYC 64C 2GHz, AMD Instinct MI250X, Slingshot-11, HPE Cray OS, HPE DOE/SC/Oak Ridge National Laboratory United States	9,066,176	1,353.00	2,055.72	24,607
3	**Aurora** - HPE Cray EX - Intel Exascale Compute Blade, Xeon CPU Max 9470 52C 2.4GHz, Intel Data Center GPU Max, Slingshot-11, Intel DOE/SC/Argonne National Laboratory United States	9,264,128	1,012.00	1,980.01	38,698
4	**JUPITER Booster** - BullSequana XH3000, GH Superchip 72C 3GHz, NVIDIA GH200 Superchip, Quad-Rail NVIDIA InfiniBand NDR200, RedHat Enterprise Linux, EVIDEN EuroHPC/FZJ Germany	4,801,344	793.40	930.00	13,088
5	**Eagle** - Microsoft NDv5, Xeon Platinum 8480C 48C 2GHz, NVIDIA H100, NVIDIA Infiniband NDR, Microsoft Azure Microsoft Azure United States	2,073,600	561.20	846.84	

Sistem paralel (El Capitan cluster)

- 9.472 CPU AMD Epyc 7713 (64 cores)
  - 606.208 core-uri
- 74 rack-cabinets
- 1 rack-cabinet are 64 server-blades
- 1 server-blade are 2 noduri
- Nodurile sunt structurate pe grupuri; grupurile sunt conectate intr-o topologie “dragonfly”







Sistem de calcul paralel

- **Informații globale disponibile**: număr de noduri ale rețelei, topologia rețelei, distribuția datelor în rețea, indexarea globală a nodurilor
  - Control asupra distribuției datelor
  - Control asupra execuției locale per nod
  - Control asupra implicării nodurilor în rețea

- **Timp de comunicație** inter-noduri neglijabil/mărginit (apropiere geografică)
  - Sincronizare: ceas fizic comun

- Topologie statică
- Probabilitatea scăzută a defectelor
- Complexitatea timp vs. complexitatea mesaj

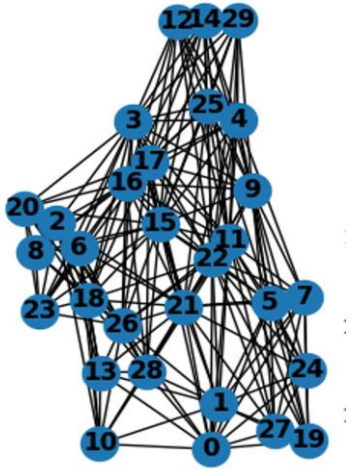


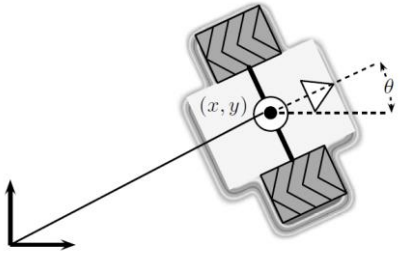
Frontier: DOE/SC/Oak Ridge National Laboratory
No.1 in Jun 2022

Arhitecturi: Peer-to-peer nestructurate

Rețele de roboți (agenți mobili), rețele de senzori, sisteme de informații, WAN, cu aplicații precum:

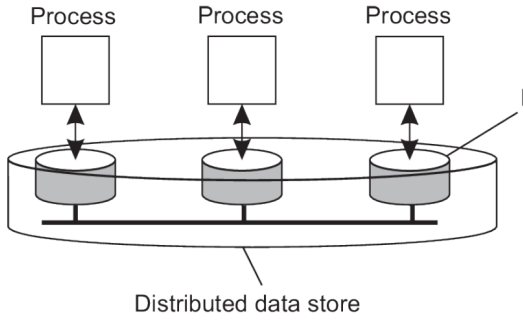
- Poziționare-localizare distribuită (e.g. GPS, rendezvous)
- Urmărire distribuită
- Achiziție distribuită
- Editare distribuită (colaborativă)
- Rezolvare distribuită de sisteme





Caracteristici specifice:

- Anonimitate: nodurile nu au un identificator unic global
- Topologia potențial variabilă în timp
- Posibile defecte pe legături sau noduri



andrei.tirascu@unibuc.ro

Sistem distribuit

Sistem distribuit = o colecție de procese autonome care comunică peste o rețea (topologie) cu următoarele proprietăți**:**

- **Fiecare nod are o „vedere" locală asupra sistemului.** Un nod al sistemului cunoaște și comunică cu propria vecinătate, neavând acces la informații globale. În general, există o separare geografică a nodurilor**.**

- **Nu există un ceas fizic comun**. Acestui aspect se datorează caracterul "distribuit" al sistemului și este cel care cauzează lipsa sincronizării între noduri.

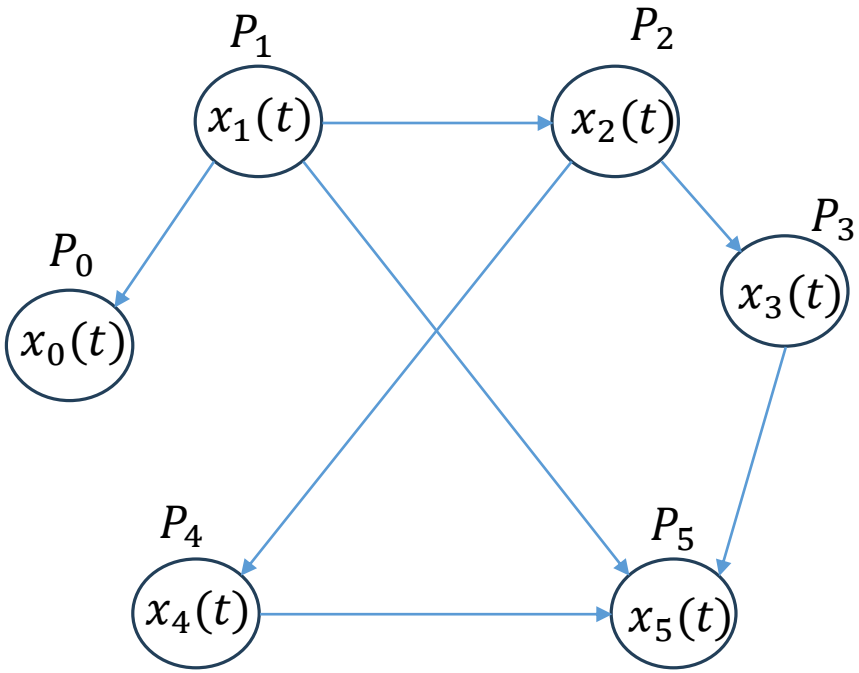
- **Nu există memorie partajată.** Proprietate care aduce necesitatea comunicației prin mesaje (în absența unui ceas global).

- **Autonomia și eterogenitatea nodurilor.** Noduri sunt "slab cuplate", au viteze diferite de execuție, au sisteme de operare diferite.

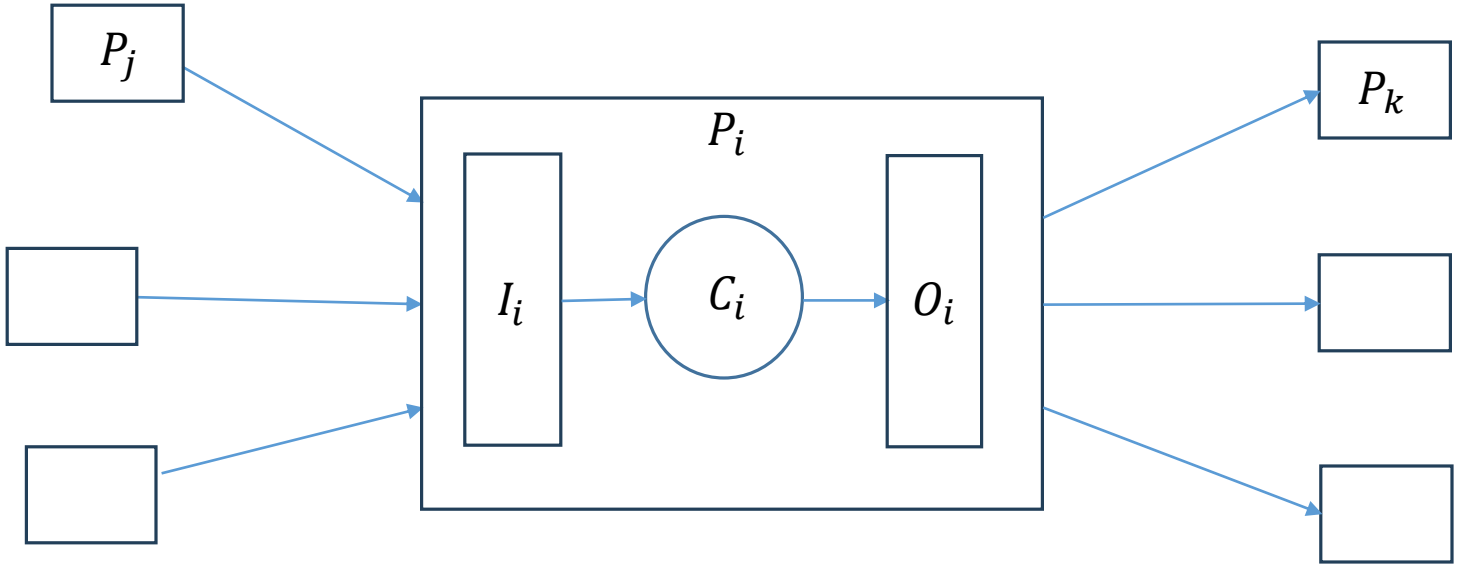
Model SD

- Noduri: $P_0, P_1, \dots, P_{n-1}$
- Rețea de comunicație $G = (V, E)$
  - $V = \{P_0, P_1, \dots, P_{n-1}\}$
  - $(i, j) \in E$ dacă există muchie între nodurile P_i și P_j
- Starea nodului P_i se exprima $x_i: \mathbb{N} \to \mathcal{D}$

- Stare nod P_i la momentul de timp t: $x_i(t)$
  - Locație (real)
  - Orientare (real)
  - Vot-Opinie (binar)

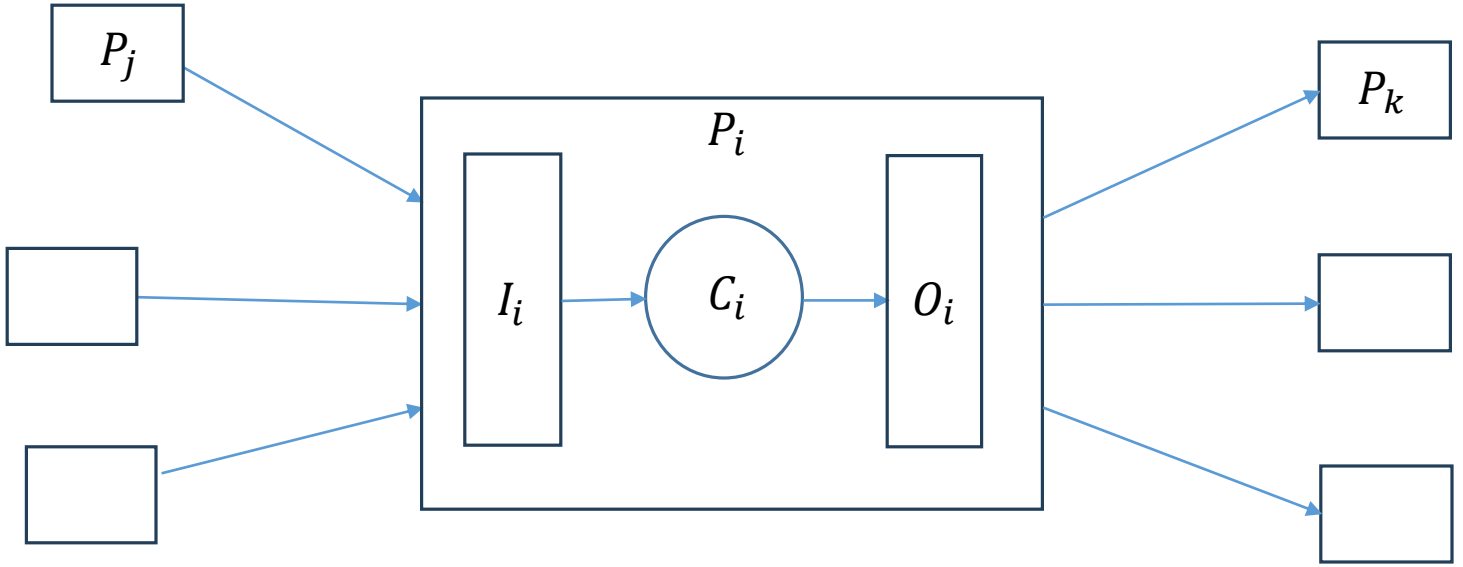


Model Proces



- Un proces primeste informatie la intrare (mesaje de la vecini), stocata in I_i
- Calculeaza noua stare pe baza info de input; in plus, depune in O_i mesaje pentru transmitere
- Transmite mesajele din O_i catre vecinii de iesire
- Transferul de mesaje este un eveniment separat reusit/esuat

Model Proces



Mai formal:

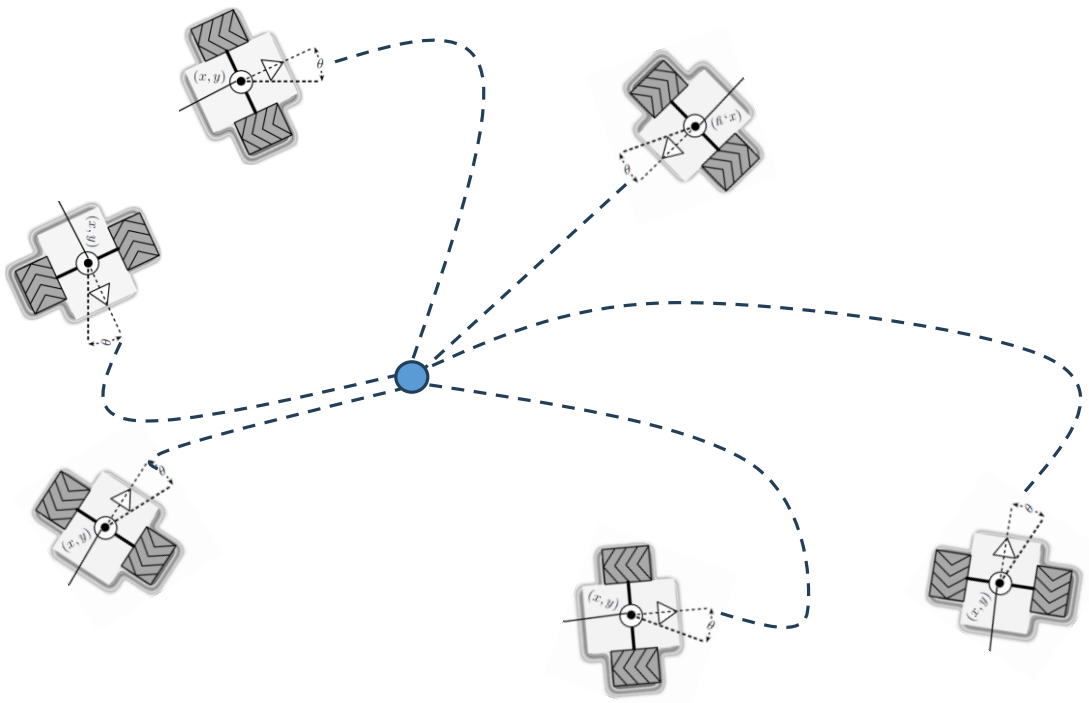
- $I_i(t), O_i(t), x_i(t)$ buffer-ele de intrare si iesire, si starea la momentul t
- Nodul i primeste mesajul m la intrare:$I_i(t+1) = I_i(t)/\{m\}$
- Calculeaza noua stare pe baza intrarii: $\mathrm{f}_i(x_i(t), m) \rightarrow (x_i(t+1), \{m_1, \ldots, m_k\})$
- Transmite mesajele de iesire: $O(t+1) = O_i(t) \cup \{m_1, \ldots, m_k\}$
- Consideram I/O de mesaje ca evenimente separate in retele nesigure

Exemplu model sincron: rendezvous

Rendezvous 2D: Considerăm o rețea de n agenți mobili $\{1,2,\cdots,n\}$ cu pozițiile inițiale $\{p_1(0), p_2(0), \cdots, p_n(0)\}$. Conduceți agenții prin intrări, calculate la nivel local, către un punct de întâlnire arbitrar, i.e. $p_i = p_j, \forall i,j \in \{1,2,\cdots,n\}$.

- Prin notația $p(t)$ extragem poziția din vectorul de stare $\mathrm{x}(t) \in X$, și considerăm modelul simplificat al agenților:
- $p_i(t+1) = p_i(t) + u_i(t)$

$$x_i(t) = \begin{bmatrix} p_x(t) \\ p_y(t) \\ \theta(t) \end{bmatrix}$$

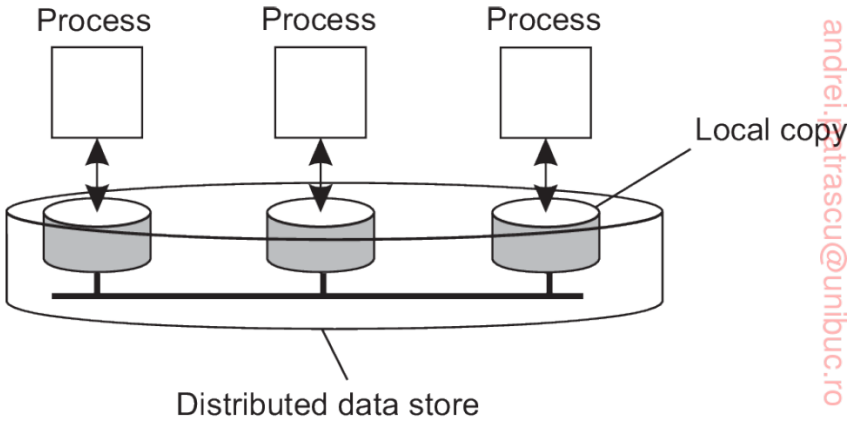


Exemplu model sincron: vot majoritar

Majority voting: Considerăm o rețea de n replici de date $\{1,2,\cdots,n\}$ cu voturi inițiale $\{x_1(0), x_2(0), \cdots, x_n(0)\}$. Aduceți replicile prin interacțiuni la nivel local, către valoarea asumată majoritar la nivel global

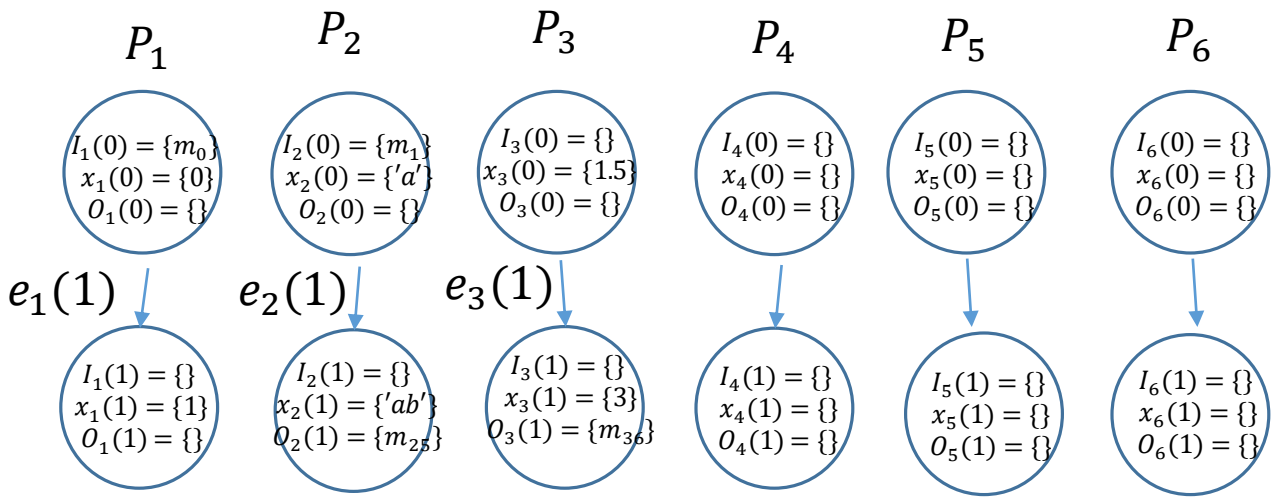
- Starea nodului denotă acțiunea de commit/abort a unei posibile tranzacții, i.e. $x_i(t) \in \{0,1\}$

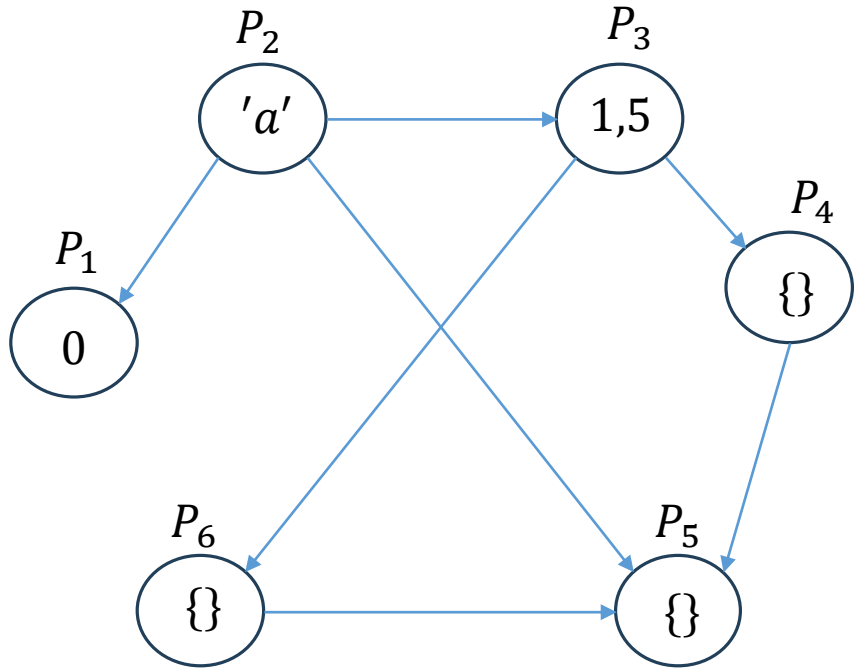
$$x_i^* = Maj\big(x(0)\big) = \begin{cases} 1, dacă\ |\{i|x_i(0) = 1\}| \geq \frac{n}{2} + 1 \\ 0, dacă\ |\{i|x_i(0) = 1\}| < \frac{n}{2} + 1 \end{cases}$$



andrei.tirascu@unibuc.ro

Modelul principal in SD sincron

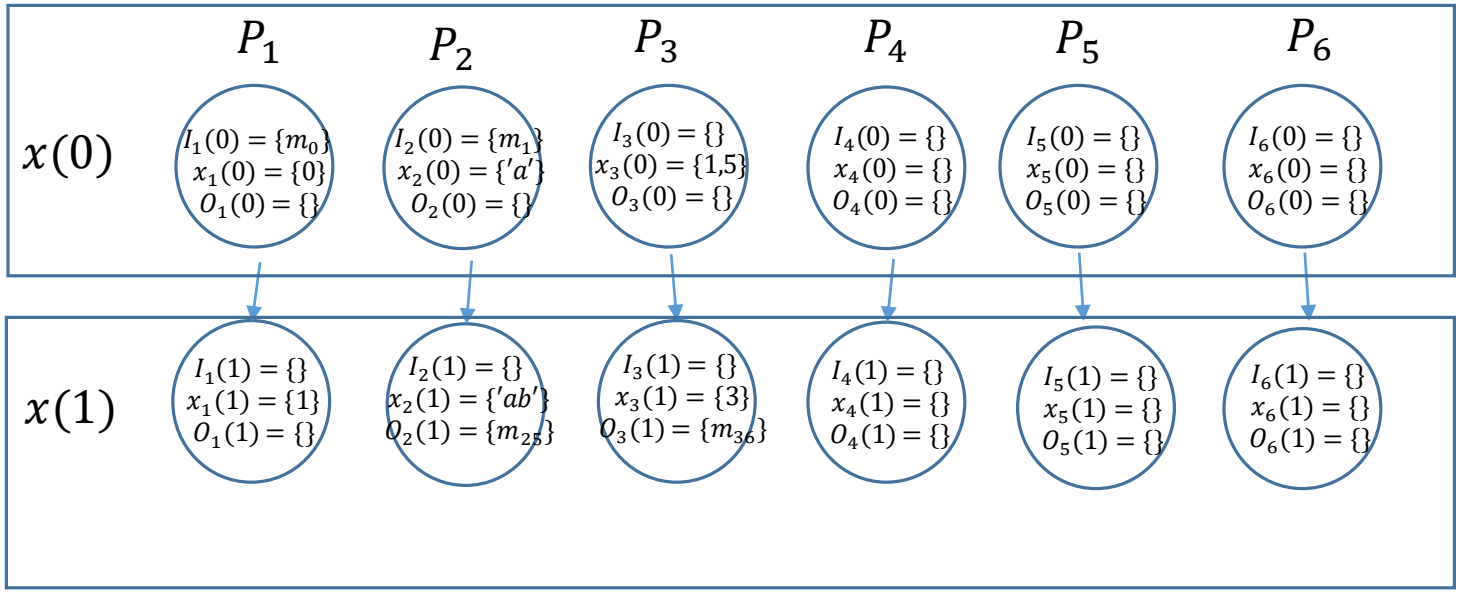


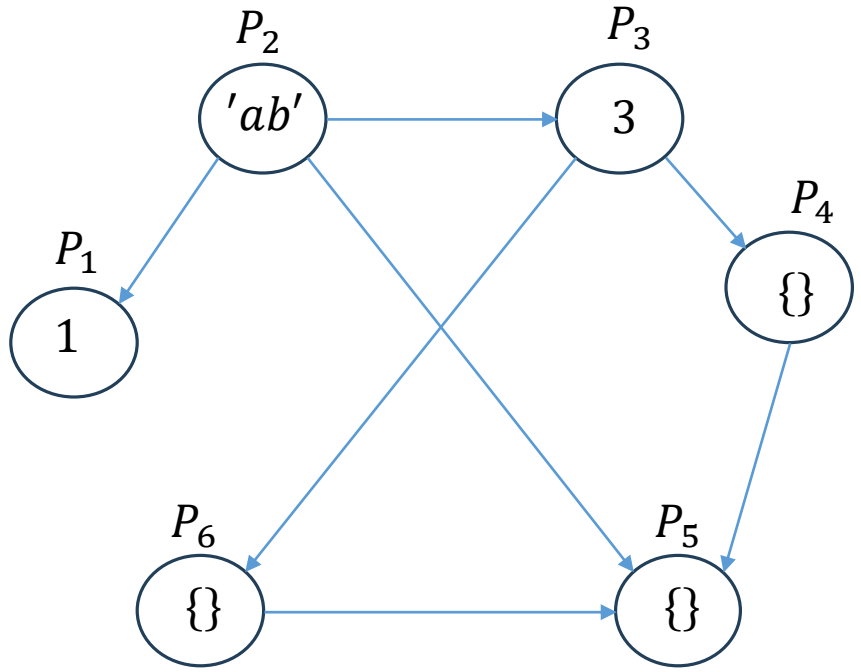


Evenimente posibile:

- Operatii locale pe baza $I_i(t)$ si $x_i(t)$: e.g. calcule numerice
- Evenimente de livrare de mesaje

Modelul principal in SD sincron





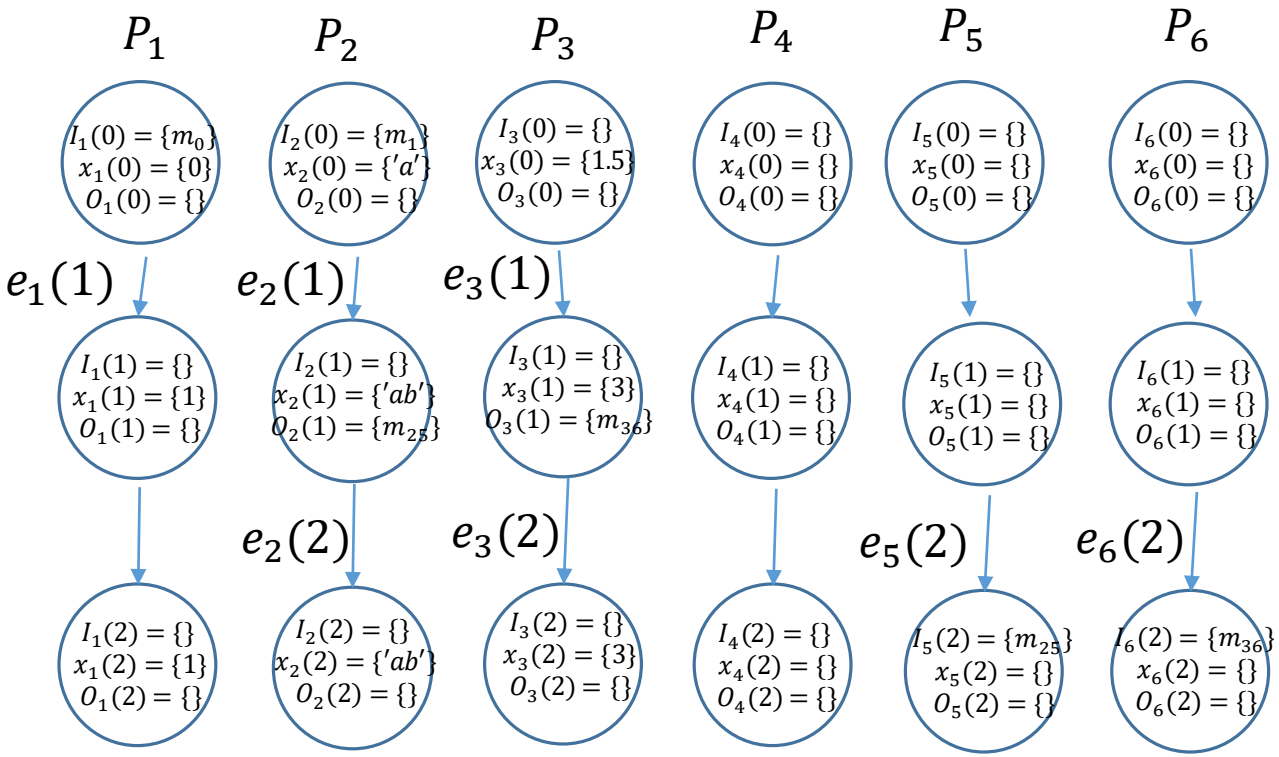
Starea globala a sistemului la momentul t :

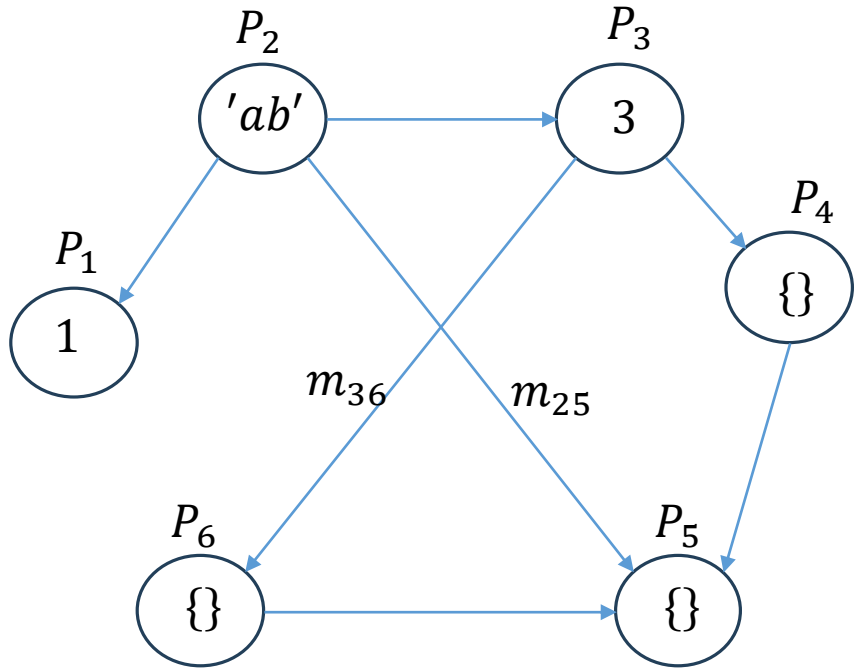
$$x(t) = \begin{bmatrix} x_1(t) \\ x_2(t) \\ x_3(t) \\ x_4(t) \\ x_5(t) \\ x_6(t) \end{bmatrix}$$

Starea initiala a sistemului:

$$x(0) = \begin{bmatrix} 0 \\ a \\ 1,5 \\ NULL \\ NULL \\ NULL \end{bmatrix} \Rightarrow \qquad x(1) = \begin{bmatrix} 1 \\ ab \\ 3 \\ NULL \\ NULL \\ NULL \end{bmatrix}$$

Modelul principal in SD sincron





Modelul principal in SD sincron

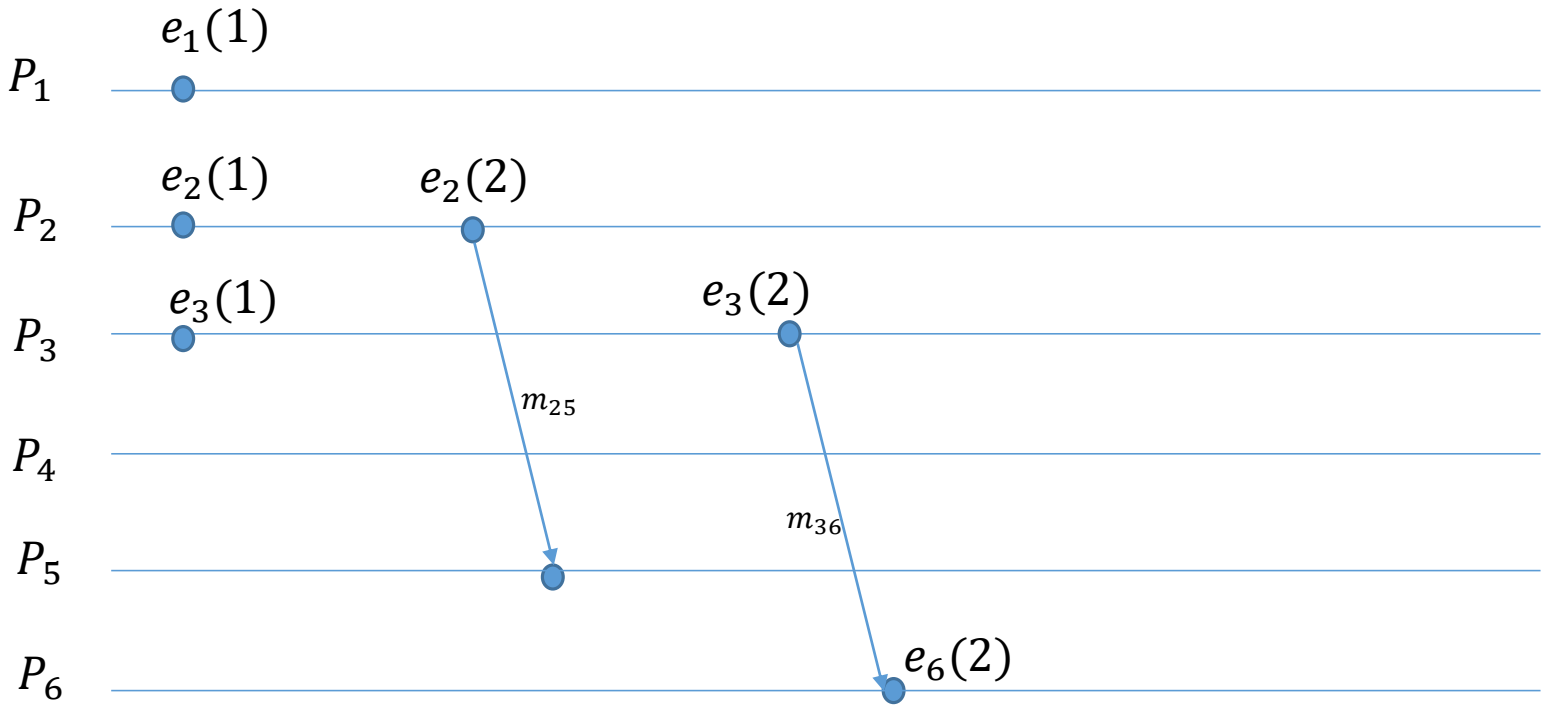


Diagrama spatiu-timp:

- Fiecare proces are propria vedere locala asupra evenimentelor
- Exprima precedenta cauzala intre evenimente

Comunicație

Modele de comunicație:

- Comunicație prin mesaje
- Mesaje persistente
- Comunicație colectivă

Comunicație

În limbajul natural (uman) exprimarea unui mesaj cuprinde:

- Dialect
- Limbă
- Cuvînt
- Alfabet

Comunicație

Premise necesare pentru comunicație:

- Sursa și destinația sunt conectate fizic
- Sursa alege alfabetul, limba etc., în care codifică mesajul
- Destinația decodifică mesajul (recunoaște alegerile sursei)

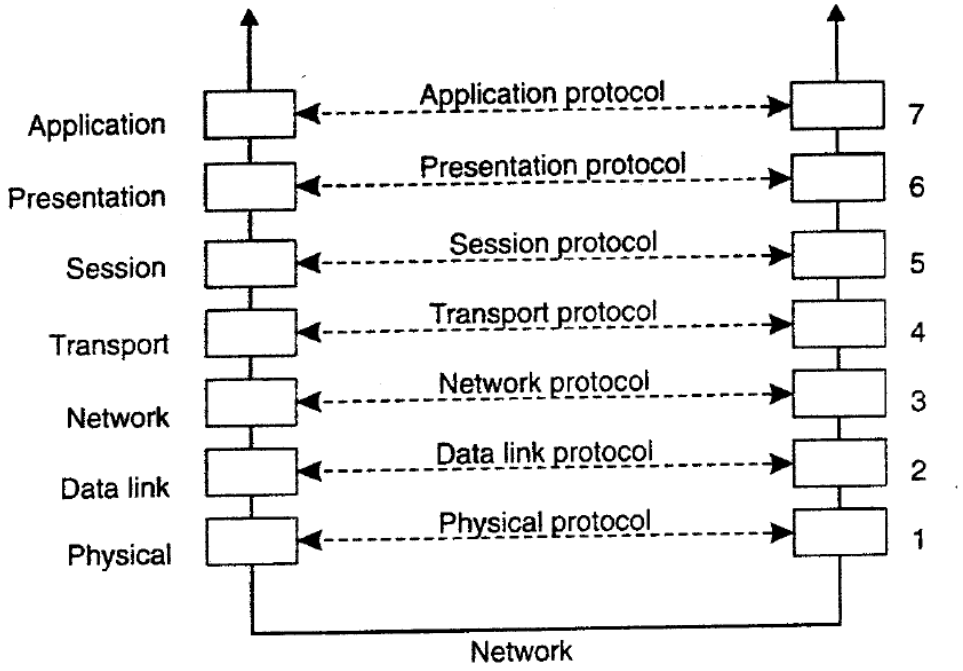
Modele de comunicație

A comunică cu B:

- A produce un mesaj
- Apelează o primitivă pentru a trimite mesajul
- B apelează o primitivă pentru a primi mesajul

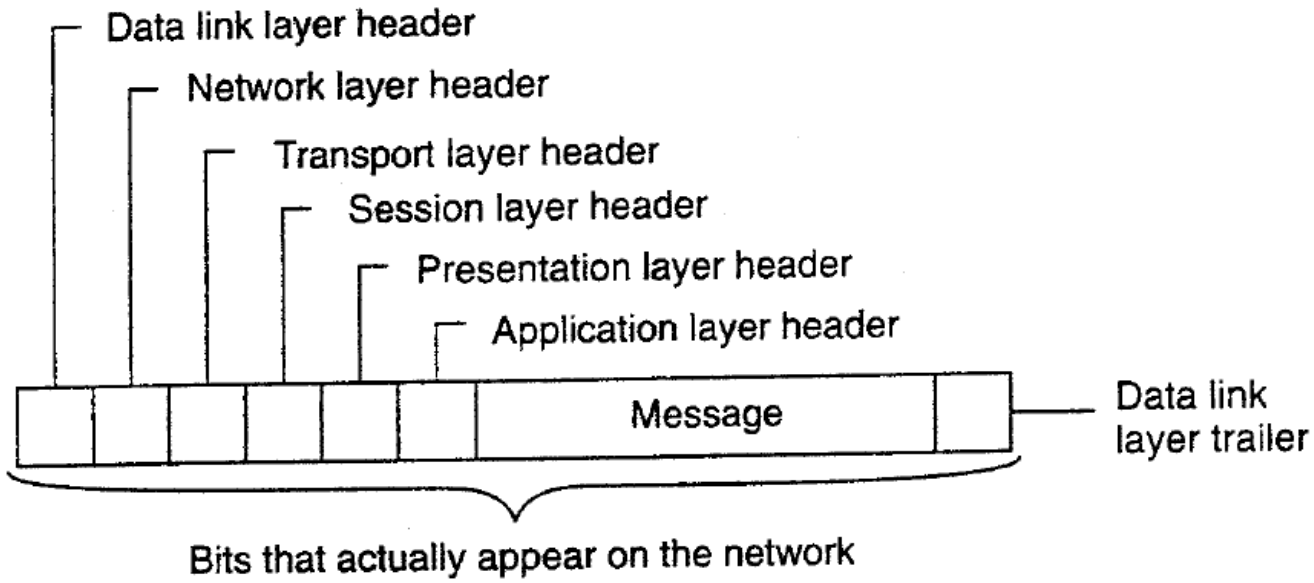
Protocoale de comunicație

Protocol = algoritm care specifică formatul, conținutul și sensul mesajelor trimise și primite de către un proces.



Protocoale de comunicație

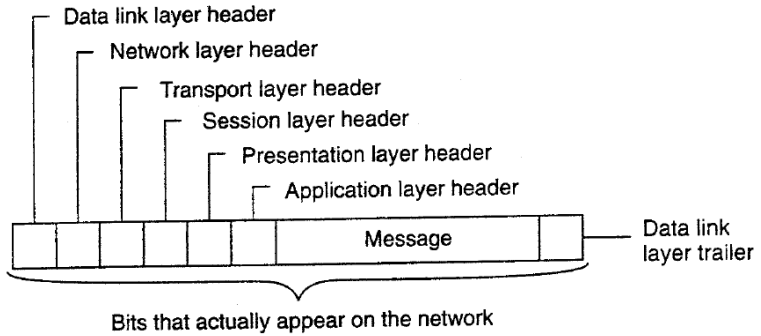
Protocol = algoritm care specifică formatul, conținutul și sensul mesajelor trimise și primite de către un proces.



Ex.: User Datagram Protocol (UDP)

Protocol de comunicație la nivelul (layer) de transport

- Fără conexiune = sursa comunică pachete către destinație fără a negocia o conexiune (implicit fără gestiunea pachetelor)
- Lightweight = nu necesită ordonare sau istoric de pachete; comunicația mesajului începe de la primul pachet
- Singurul mecanism de siguranță este *checksum*, nu există verificarea succesului transmisiei.
- Folosit în aplicațiile de streaming, broadcasting



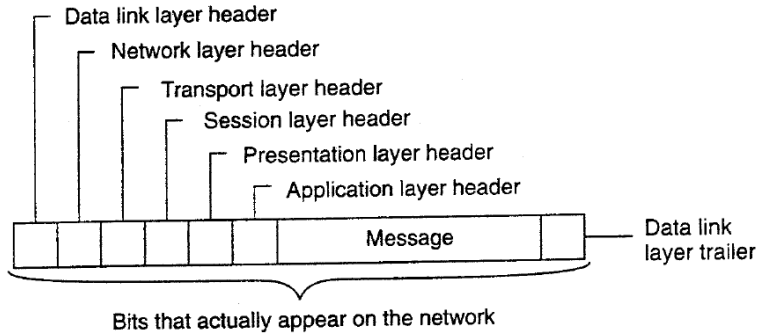
Ex.: User Datagram Protocol (UDP)

Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Source Port																Destination Port															
4	32	Length																Checksum															
8	64	Data																															
12	96																																
⋮	⋮																																

Ex.: Transmission Control Protocol (TCP)

Protocol de comunicație la nivelul (layer) de transport

- Orientat pe conexiune = înainte de interschimbarea de mesaje dintre sursă și destinație, se realizează o conexiune.
- Heavyweight = primele 3 pachete necesare pentru conexiune prin socket.
- Mecanisme de ACK, retransmisie și time-out
- Cuprinde trei etape:
  - Stabilire conexiune
  - Transfer date
  - Oprire conexiune
- Divide mesajul în porțiuni și creează segmente de dimensiuni fixe

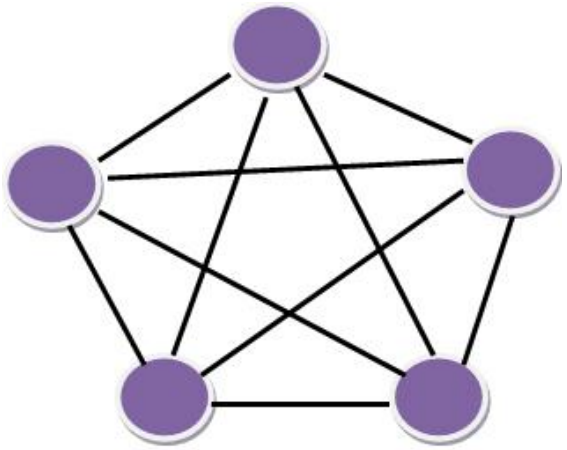


Ex.: Transmission Control Protocol (TCP)

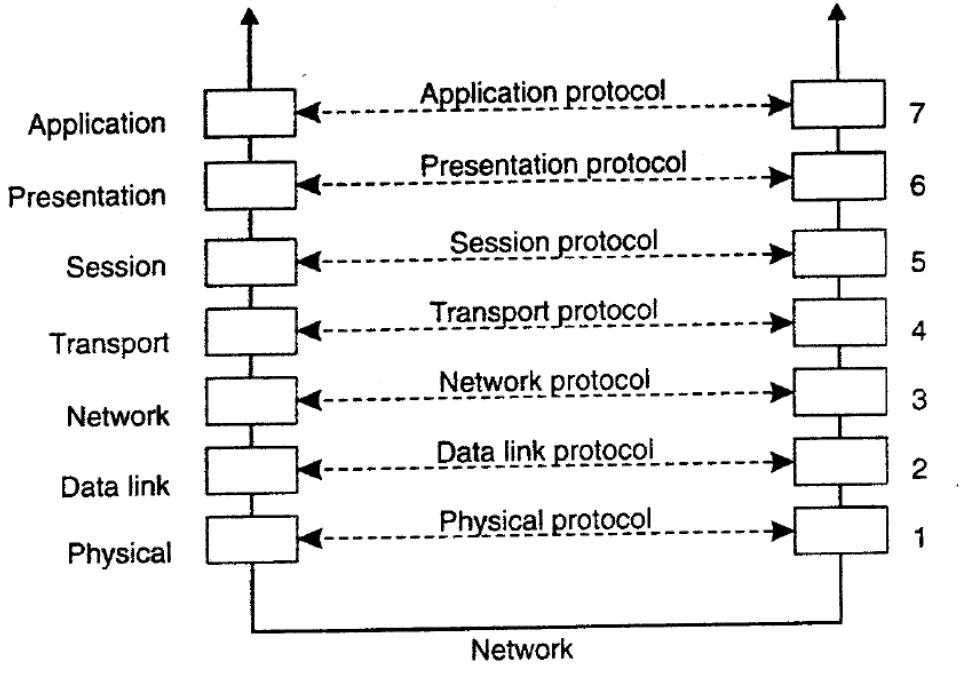
Offset	Octet	0								1								2								3							
Octet	Bit	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
0	0	Source Port																Destination Port															
4	32	Sequence Number																															
8	64	Acknowledgement Number (meaningful when ACK bit set)																															
12	96	Data Offset				Reserved				CWR	ECE	URG	ACK	PSH	RST	SYN	FIN	Window															
16	128	Checksum																Urgent Pointer (meaningful when URG bit set)[18]															
20	160	[Options] If present, Data Offset will be greater than 5. Padded with zeroes to a multiple of 32 bits, since Data Offset counts words of 4 octets.																															
⋮	⋮																																
56	448																																
60	480	Data																															
64	512																																
⋮	⋮																																

Modele de comunicație

1. *Comunicație prin mesaje*
2. Comunicație prin mesaje persistente
3. Comunicație colectivă

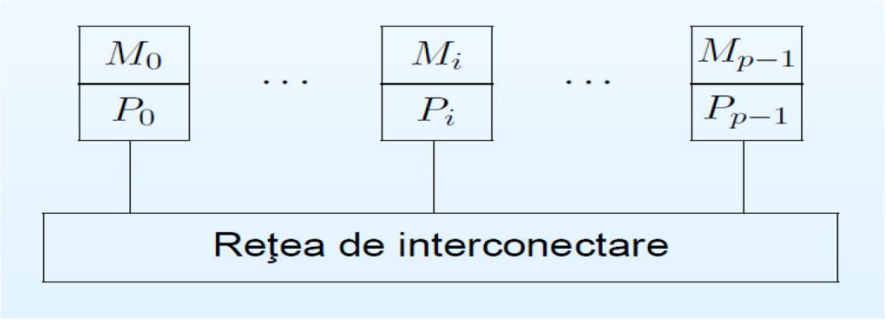


http://4.bp.blogspot.com/-LLvGw5z-_4/UMadzzwP7tI/AAAAAAAAAHY/2AxHYokPHzk/s1600/all-channel.jpg



Comunicația prin mesaje: Modelul SPMD

- MIMD cu memorie distribuita
- Paradigma **SPMD** (Single Program Multiple Data): toate procesoarele execută același program, dar fiecare utilizează un set propriu de date.
- În general, execuția programului nu este sincronă;
- Deși toate procesoarele văd același program, instrucțiunile executate nu sunt identice



Modelul SPMD

- Procesoarele sunt numerotate 0, ..., p
- Numerotarea nu este statică, ci se realizează la momentul execuției.
- Există primitive care returnează adresa unui procesor (e.g. MPI_rank)
- Procesoarele pot executa instrucțiuni diferite în funcție de adresa lor

1. rank ← adresa proprie
2. **dacă** rank = 0 **atunci**
    1. a ← 1
3. **altfel**
    1. a ← 0

Modelul SPMD - variabile

- O variabilă a unui program SPMD este multiplicată (de p ori) : fiecare procesor deține un exemplar, asupra căruia are control complet.
- Un procesor nu poate modifica variabile din memoria altui procesor.
- Putem interpreta variabila a ca un vector cu p elemente: fiecare procesor P_i deține componenta i a vectorului. Cu toate acestea i reprezintă un index global al datelor din a.
- Programul inițializează a cu 0 pe toate componentele, cu excepția primei componente (care este 1).

Modelul SPMD - problemă

Inițializați un vector a de dimensiune n cu zero, mai puțin elementul a_m, care să fie 1.

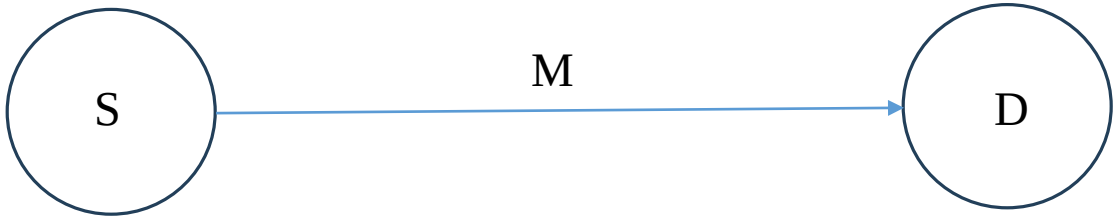
- Cum distribuim vectorul a celor p procesoare?

- Cum facem efectiv inițializarea?

Modelul de comunicație prin mesaje

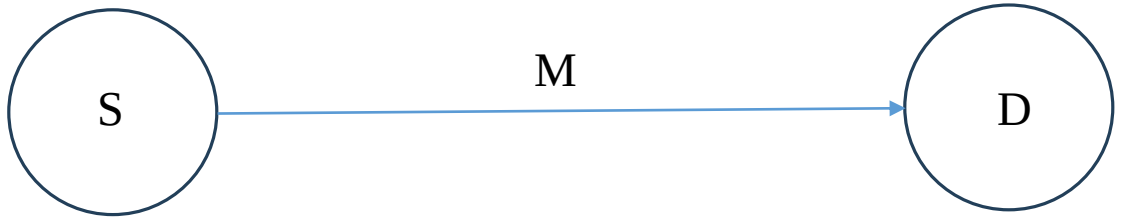
- Singura modalitate de comunicare între procesoare este transmiterea de mesaje

- **Operația de bază**:



Modelul de comunicație prin mesaje

- **Operația de bază**:

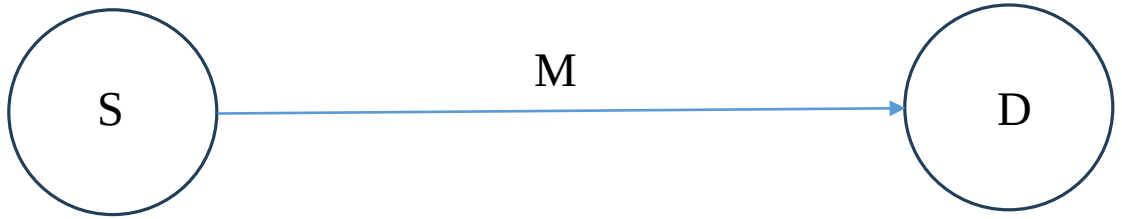


- Procesorul sursă transmite prin rutina **send**
- Procesorul destinație recepționează prin rutina **recv**

- Sintaxă generală:
  - send(date, dest)
  - recv(date, sursa)
- date = locație (buffer) din care se preiau/depun mesajele transmise
- sursa/dest = adresa procesorului cu care se comunică

MP - corectitudine

- **Operația de bază**:



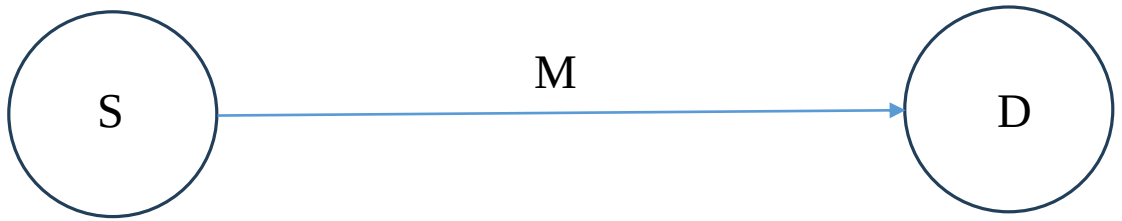
- Orice operație de **send** trebuie însoțită de una de **recv**
- Per ansamblu, vom avea perechi **send-recv**

Exemplu: Procesorul i transmite un mesaj M vecinilor de la stânga, respectiv dreapta (pe o topologie inel)

1. **dacă** rank = i **atunci**
    1. send(M, $(i+1) mod\ p$);
    2. send(M, $(i-1) mod\ p$);
2. **Altfel dacă** rank =$(i+1) mod\ p$ **atunci** recv(M, i);
3. **Altfel dacă** rank =$(i-1) mod\ p$ **atunci** recv(M, i);

MP - sincronizare

- **Operația de bază**:

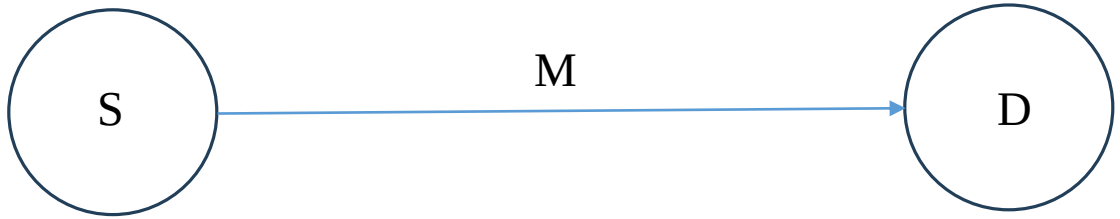


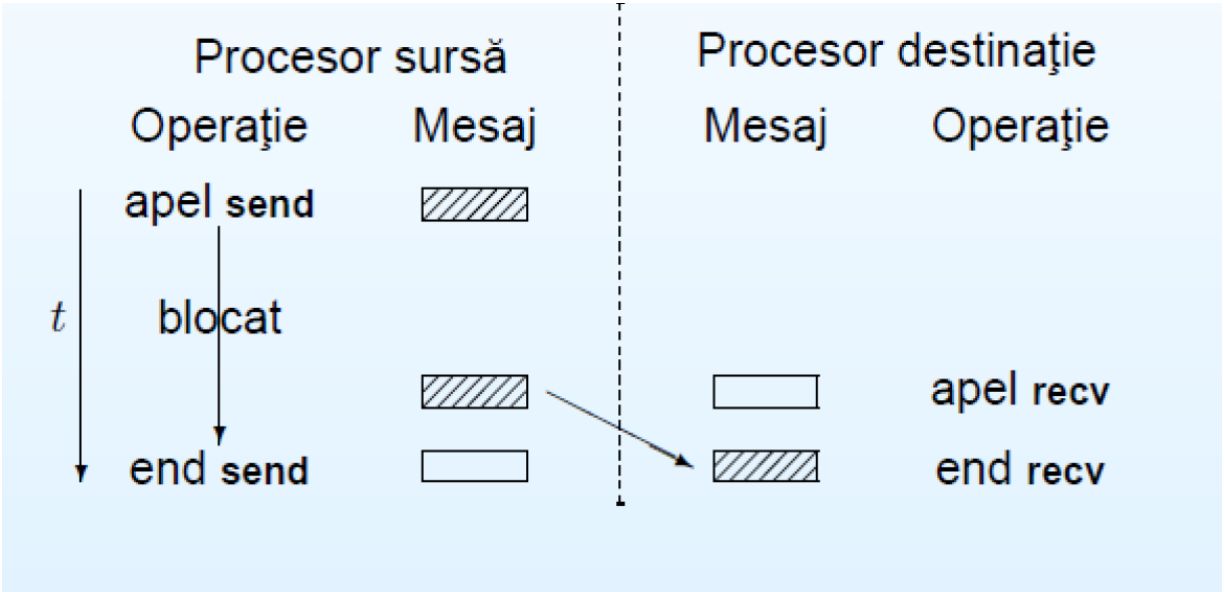
- Momentul apelului primitivei **send** este în general diferit față de momentului apelului **recv**

- Ce se întâmplă din momentul apelului primei primitive până la finalizarea comunicației?
  - Răspunsuri posibile: (i) așteaptă (**comunicație blocantă**); (ii) poate executa alte operații (**comunicație non-blocantă**)

MP – comunicație blocantă

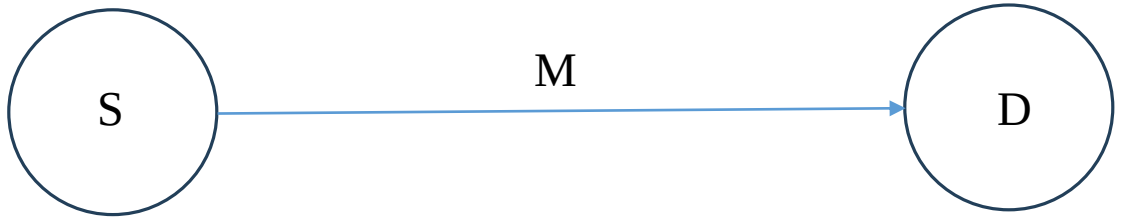
- **Operația de bază**:

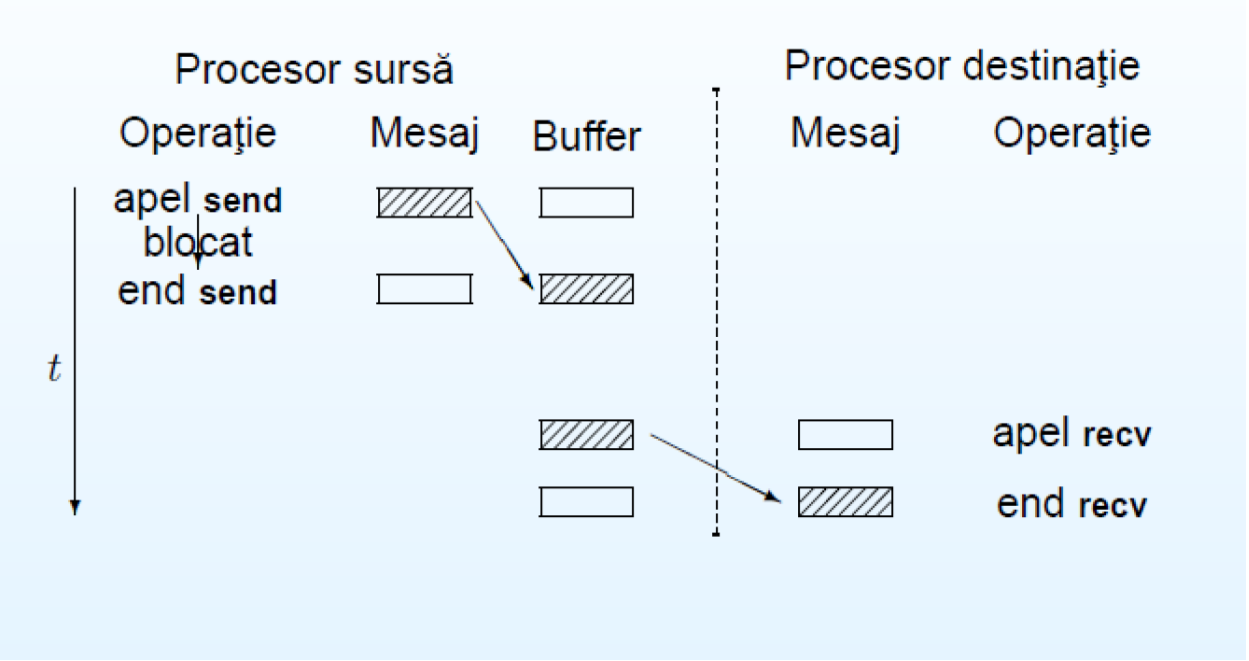




MP – comunicație blocantă prin buffer

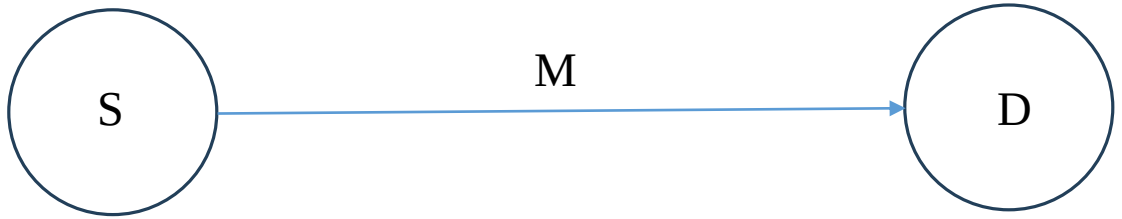
- **Operația de bază**:

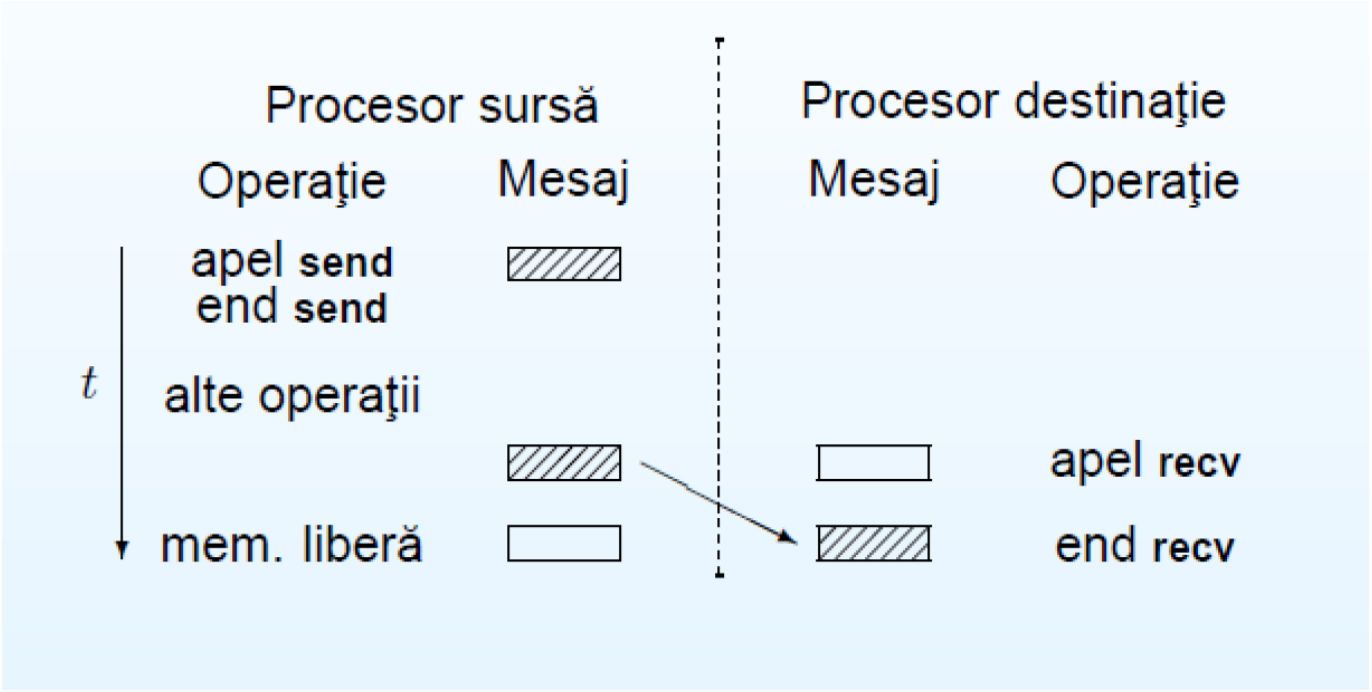




MP – comunicație non-blocantă

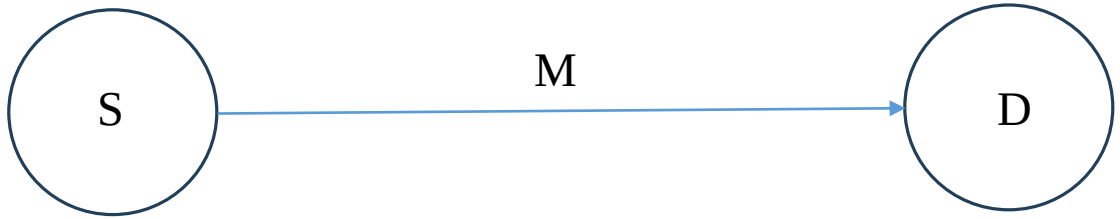
- **Operația de bază**:





MP – comunicație non-blocantă

- **Operația de bază**:



Alte primitive:

- **Așteaptă (Wait) –** așteptarea terminării comunicației

Utilizare:

1. **send**(*date*, dest);
2. execută operații care nu modifică date;
3. **așteaptă** terminarea **send**;
4. modifică date;

MP – comparație

- Comunicația non-blocantă asigură o ocupare mai bună a procesoarelor

- Erorile de programare:
  - Comm blocantă: blocarea execuției
    - send(m1, (rank + 1) mod p)
    - recv(m2, (rank - 1) mod p)

  - Comm non-blocantă: alterarea zonei de memorie după apelul **send**

Standardul MPI (Message-Passing Interface)

- Standard care descrie primitivele de comunicație în paradigma SPMD
- Implementări: OpenMPI, mpich2 etc.
- O rutină MPI se execută în cadrul unui grup de procesoare (comunicator)
- Într-un comunicator procesoarele sunt numerotate $\{0, \ldots, p-1\}$

C:

int MPI_Comm_rank(MPI_Comm com, int * my_rank)

int MPI_Comm_size(MPI_Comm com, int *p)

Python:

int comm.Get_rank()

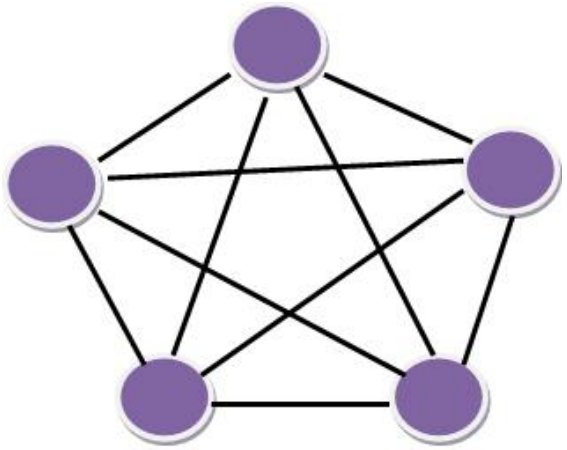
int comm.Get_size()

MPI primitives

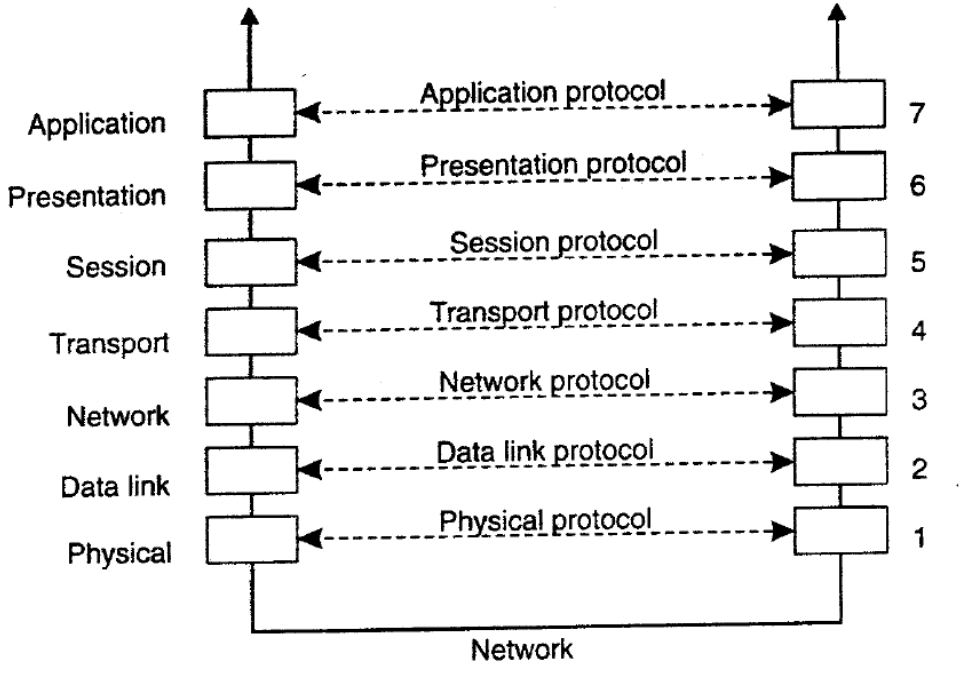
Primitive	Meaning
MPI_bsend	Append outgoing message to a local send buffer
MPI_send	Send a message and wait until copied to local or remote buffer
MPI_ssend	Send a message and wait until receipt starts
MPI_sendrecv	Send a message and wait for reply
MPI_isend	Pass reference to outgoing message, and continue
MPI_issend	Pass reference to outgoing message, and wait until receipt starts
MPI_recv	Receive a message; block if there is none
MPI_irecv	Check if there is an incoming message, but do not block

Modele de comunicație

1. Comunicație prin mesaje
2. *Comunicație prin mesaje persistente*
3. Comunicație colectivă



http://4.bp.blogspot.com/-LLvGw5z-_4/UMadzzwP7tI/AAAAAAAAAHY/2AxHYokPHzk/s1600/all-channel.jpg



Comunicație prin mesaje persistente (MQS)

Message Queuing Systems = sistem în care aplicațiile comunică prin inserarea de mesaje în cozi specifice; "O abstractizare a căsuței poștale"

- Sursa (producer) are garanția că mesajul său va fi eventual inserat în coada destinatarului (receiver)
- Nu avem garanții despre momentul când (sau dacă) mesajul va fi citit
- Sursa și destinatarul execută complet independent unul de celălalt.



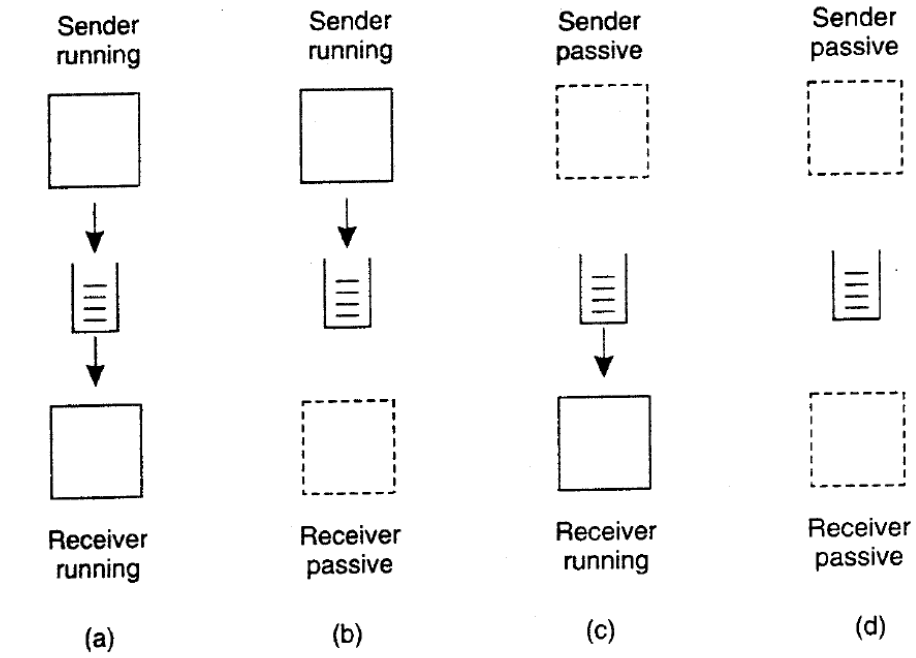


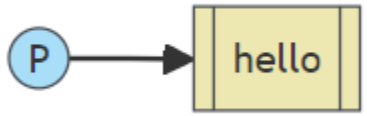
Figure 4-17. Four combinations for loosely-coupled communications using queues.

MQS primitives

Primitive	Meaning
Put	Append a message to a specified queue
Get	Block until the specified queue is nonempty, and remove the first message
Poll	Check a specified queue for messages, and remove the first. Never block
Notify	Install a handler to be called when a message is put into the specified queue

Figure 4-18. Basic interface to a queue in a message-queuing system.

Rabbit MQ example - sender




```
#!/usr/bin/env python
import pika

connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
channel = connection.channel()
```

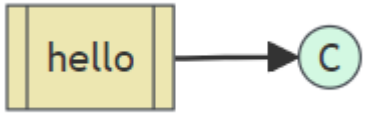
- Client RabbitMQ ⇒ modul Python pika
- Asigurăm existența cozii în care scriem mesajele (cu numele ‘hello’)

```
channel.queue_declare(queue='hello')
```

```
channel.basic_publish(exchange='',
                      routing_key='hello',
                      body='Hello World!')
print(" [x] Sent 'Hello World!'")
```

```
connection.close()
```

Rabbit MQ example - receiver




```
#!/usr/bin/env python
import pika

connection = pika.BlockingConnection(pika.ConnectionParameters('localhost'))
channel = connection.channel()
```

- Se realizează conexiunea identic
- Dacă nu cunoaștem a priori existența cozii create de sursă, asigurăm existența cozii în același fel

```
channel.queue_declare(queue='hello')
```

```
def callback(ch, method, properties, body):
    print(f" [x] Received {body}")
```

```
channel.basic_consume(queue='hello',
                      auto_ack=True,
                      on_message_callback=callback)
```

```
print(' [*] Waiting for messages. To exit press CTRL+C')
channel.start_consuming()
```

```
if __name__ == '__main__':
    try:
        main()
    except KeyboardInterrupt:
        print('Interrupted')
        try:
            sys.exit(0)
        except SystemExit:
            os._exit(0)
```

Rabbit MQ example

Execuție pe receiver:

```
python receive.py
# => [*] Waiting for messages. To exit press CTRL+C
```

```
# => [*] Waiting for messages. To exit press CTRL+C
# => [x] Received 'Hello World!'
```

Execuție pe sender:

```
python send.py
# => [x] Sent 'Hello World!'
```

https://www.rabbitmq.com/tutorials

References

[2] A. D. Kshemkalyani and M. Singhal, *Distributed Computing: Principles, Algorithms, and Systems*, Cambridge University Press, 2008.

[4] M. Raynal, *Distributed Algorithms for Message-Passing Systems*, Springer-Verlag, 2013.